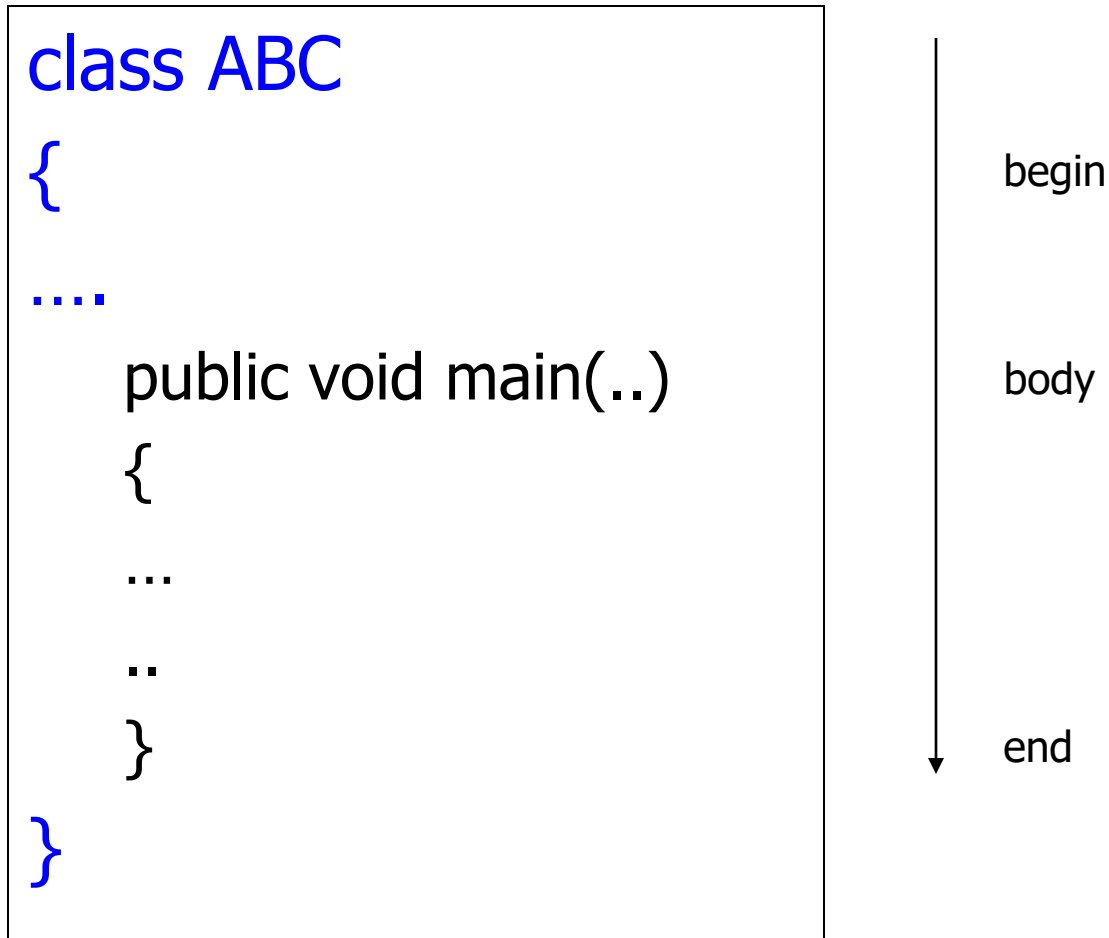


Multithreaded Programming using Java Threads

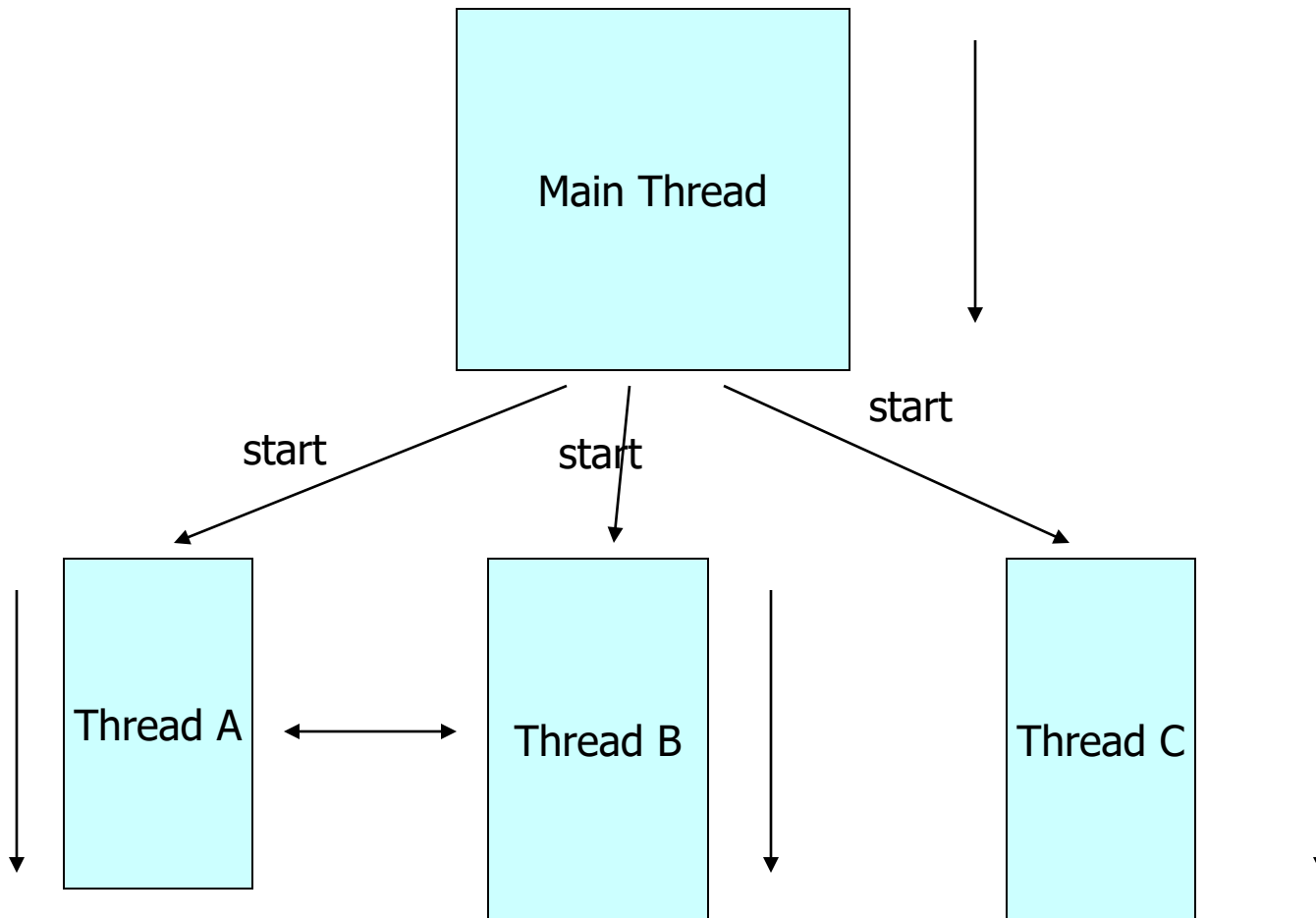
Our Goals

- Appreciate the (increasing) importance of parallel programming
- Understand fundamental concepts:
 - Parallelism, threads, multi-threading, concurrency, locks, etc.
- See some basics of this in Java

A single threaded program



A Multithreaded Program



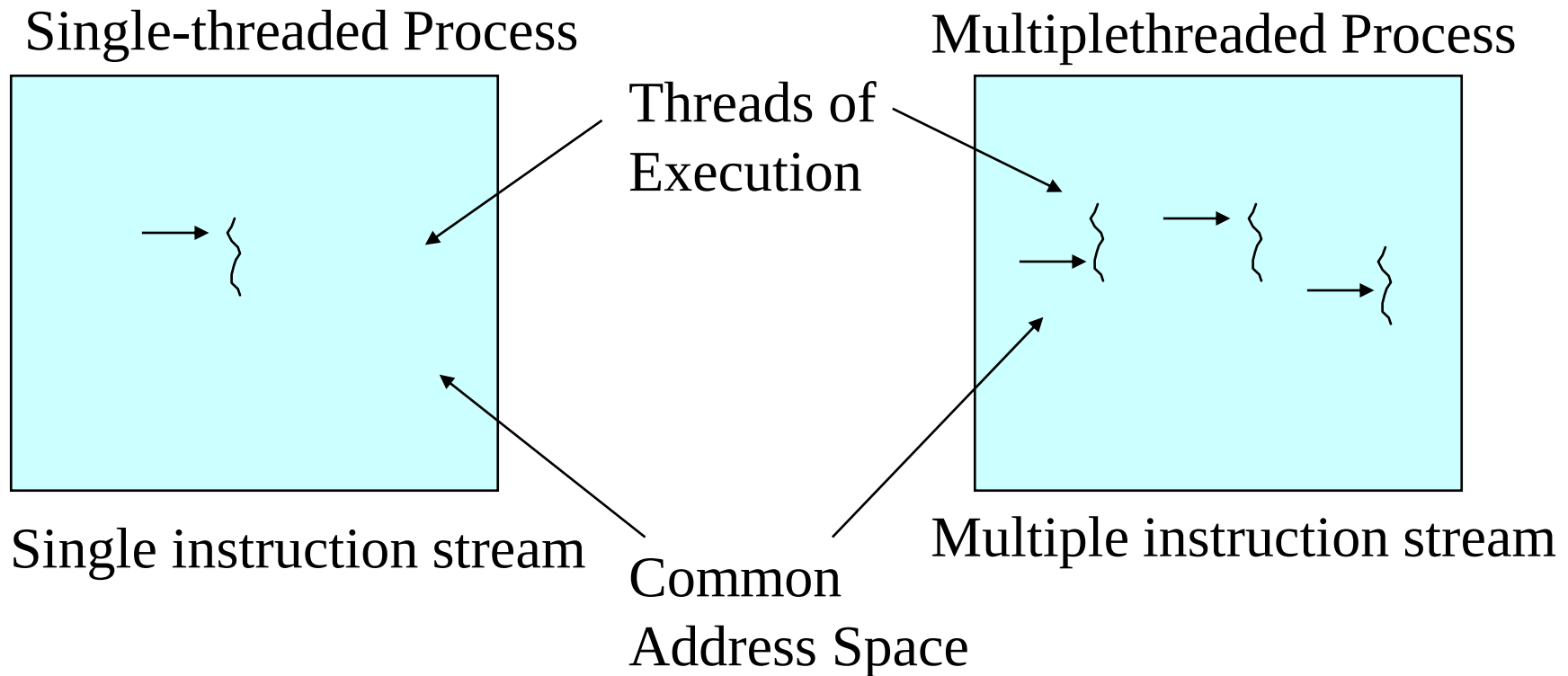
Threads may switch or exchange data/results

Why use Multithreading?

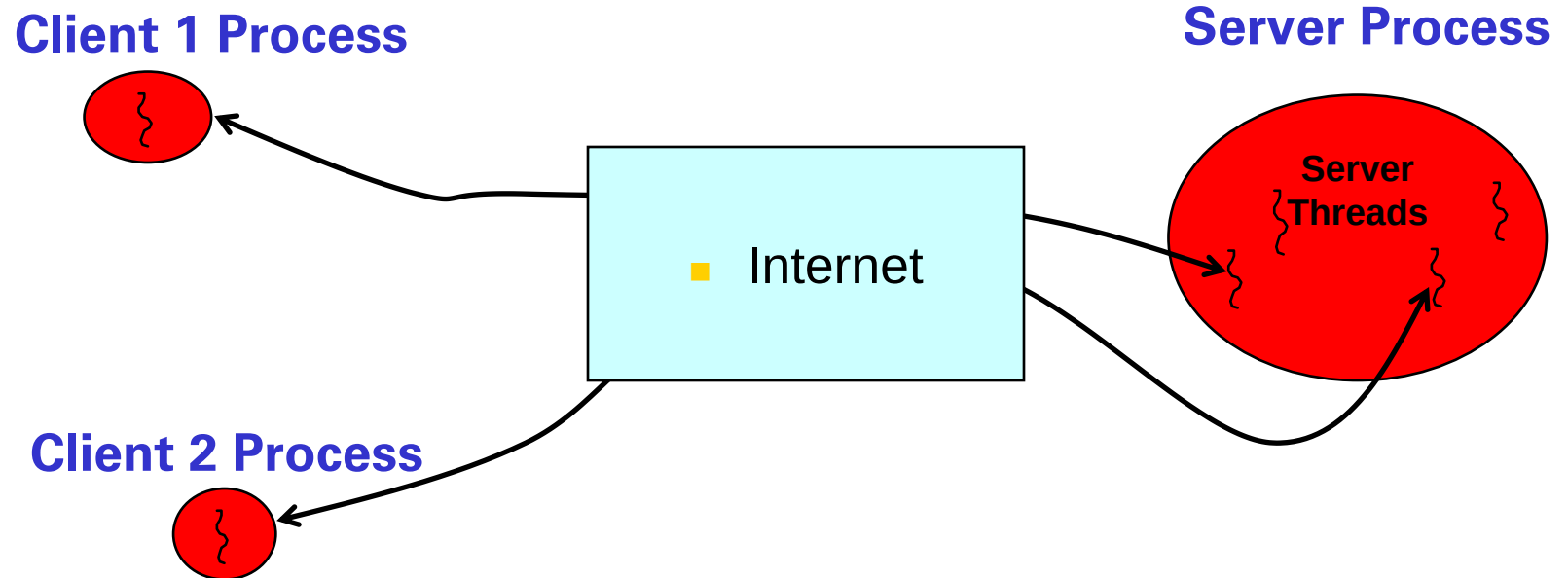
- In a **single threaded application**, one thread of execution do everything
 - If an application has several tasks to perform, tasks will be performed sequentially
 - A single task which requires a lot of processing can make the entire application in waiting.
- In a **multithreaded application**, each task can be performed by a separate thread
 - If one thread is executing for long, it does not make the entire application wait for it to finish.
- If a multithreaded application is being executed on a system that has multiple processors, the OS may execute separate threads simultaneously on separate processors.

Single and Multithreaded Processes

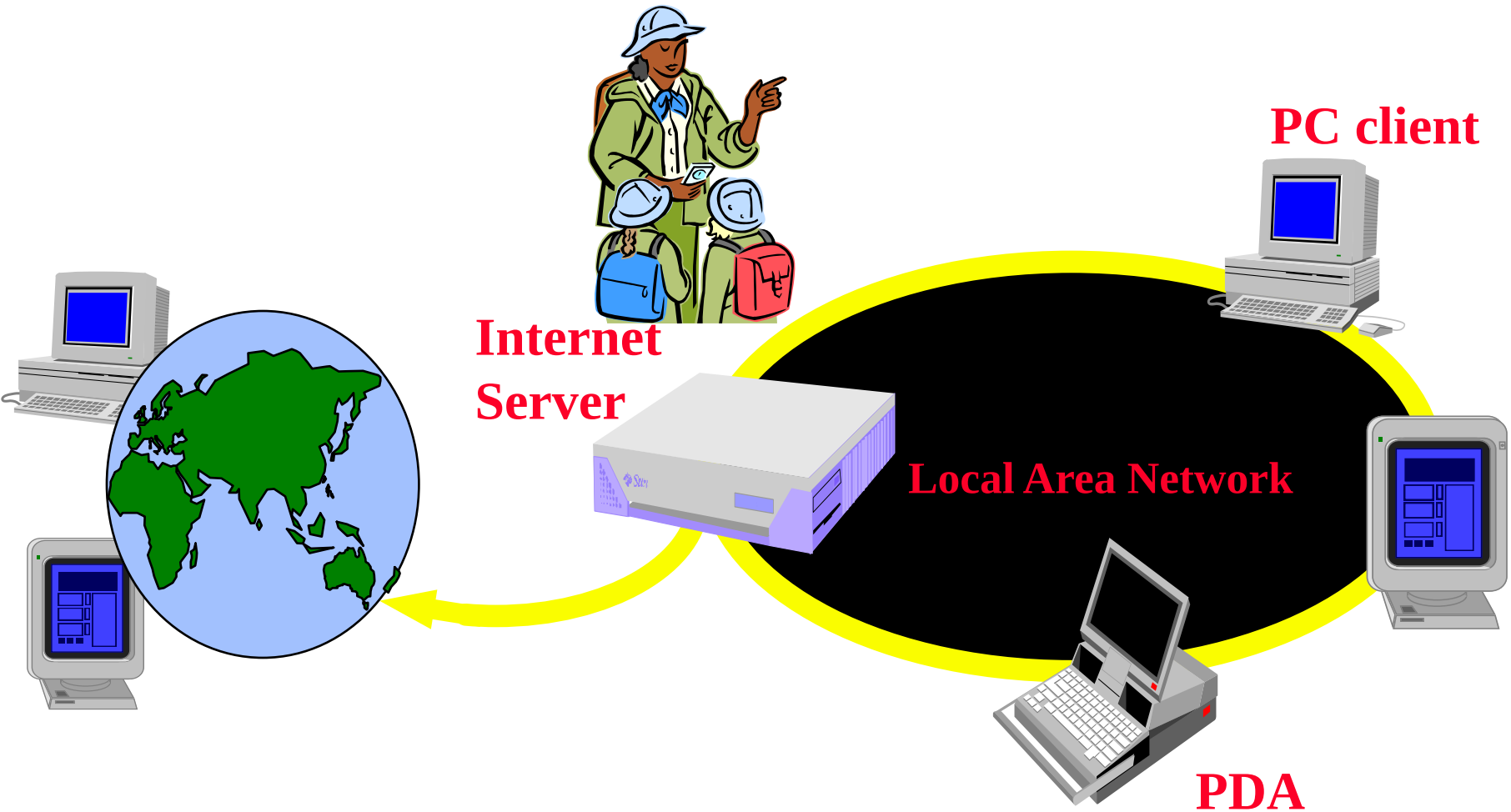
threads are light-weight processes within a process



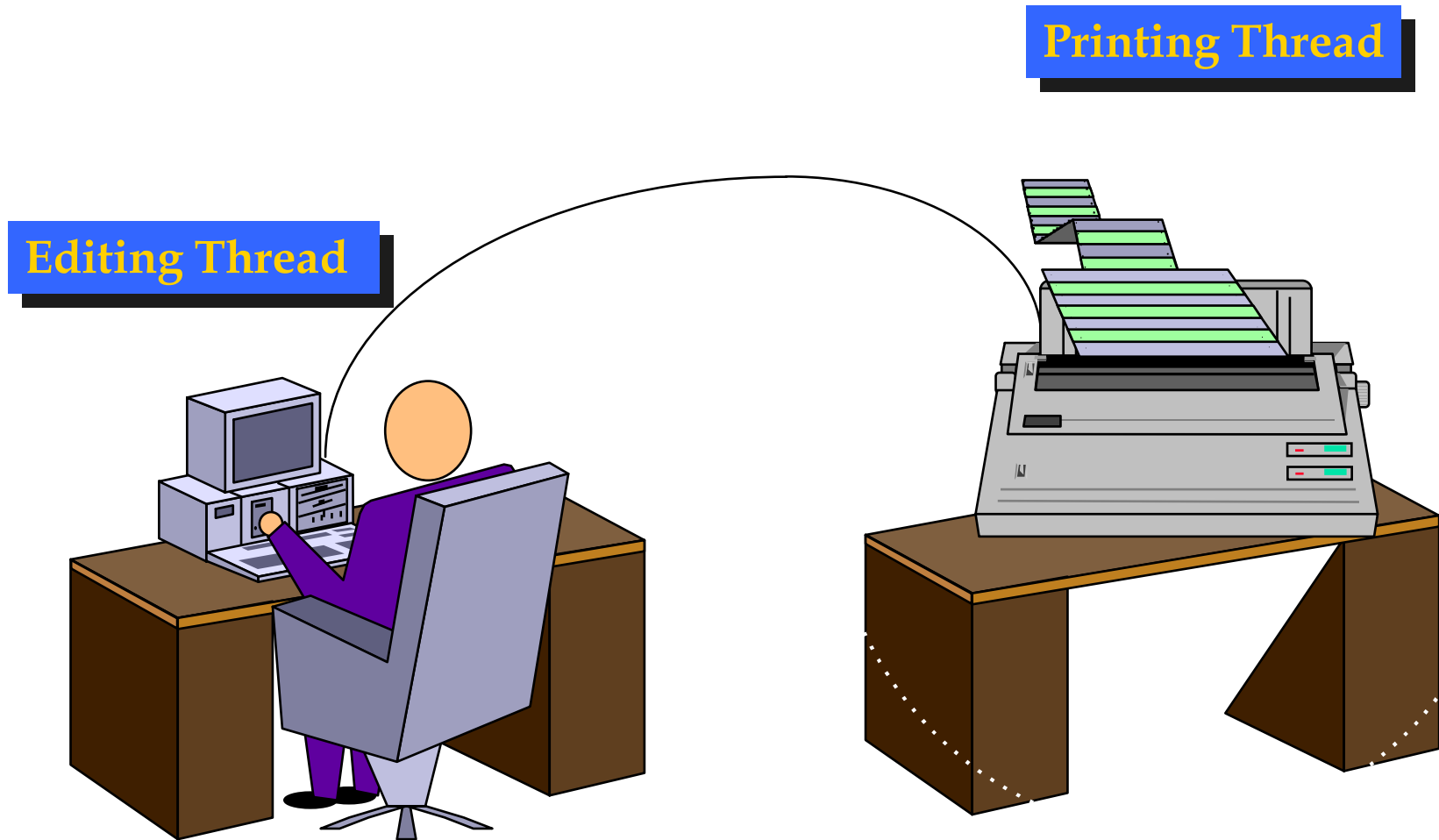
Multithreaded Server: For Serving Multiple Clients Concurrently



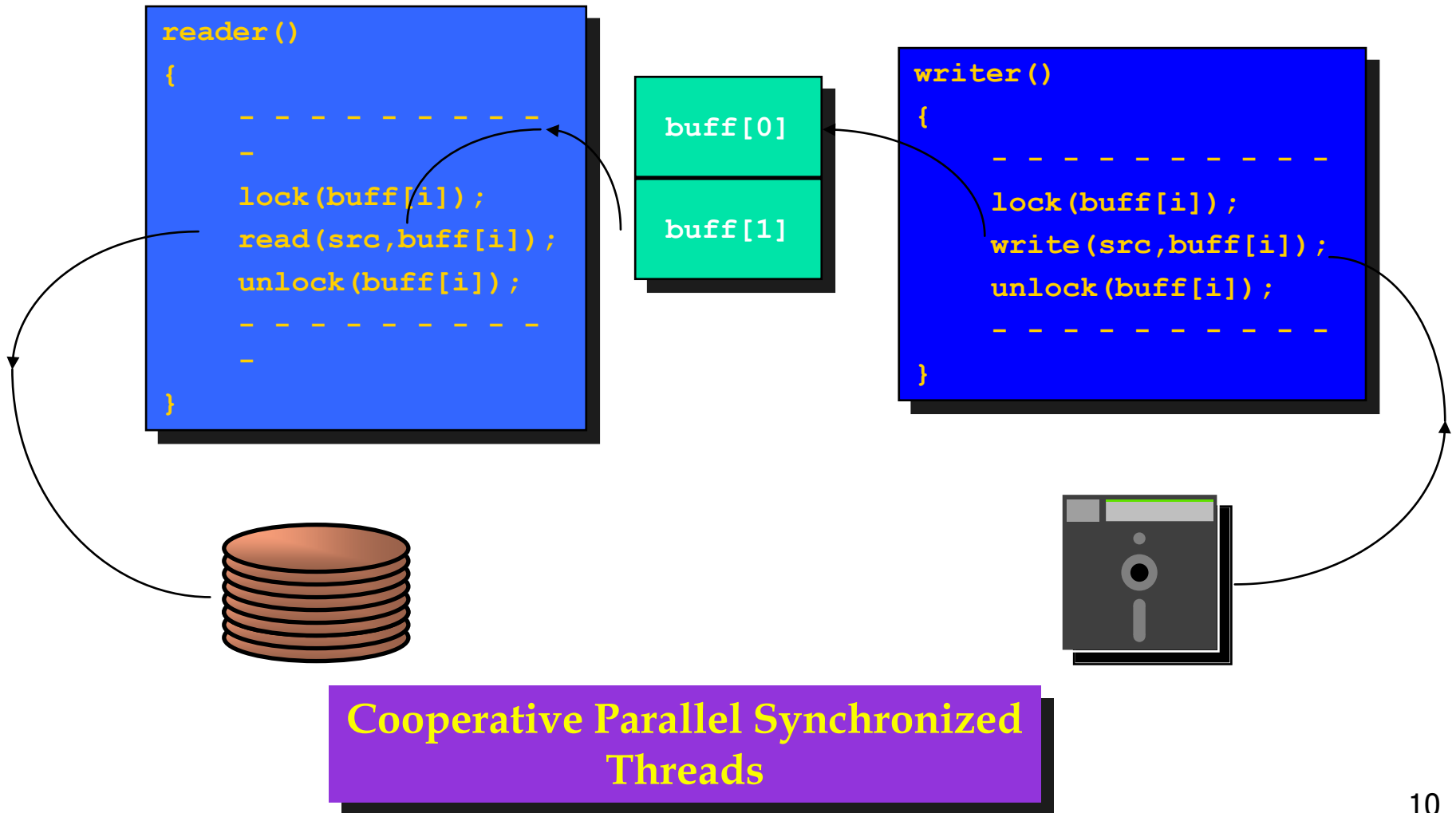
Web/Internet Applications: Serving Many Users Simultaneously



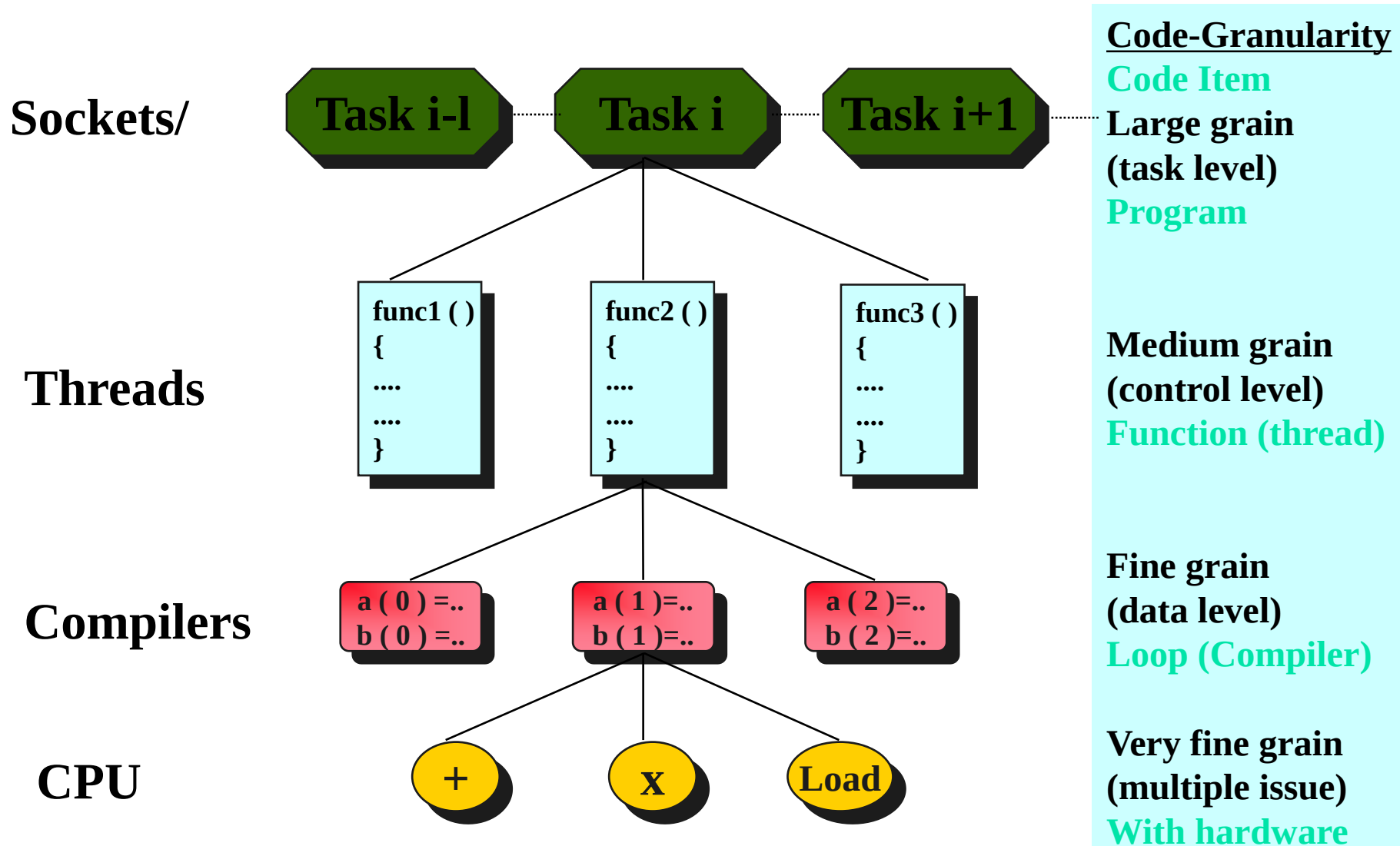
Modern Applications need Threads (ex1): Editing and Printing documents in background.



Multithreaded/Parallel File Copy



Levels of Parallelism



What are Threads?

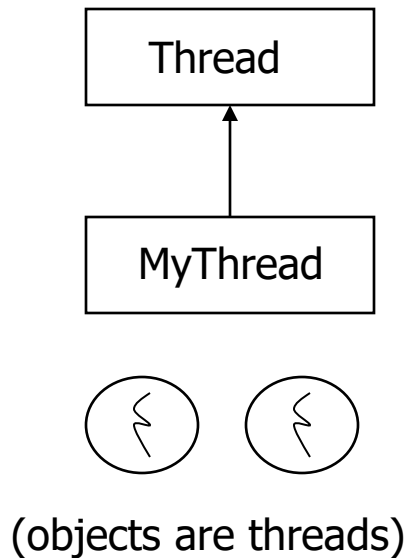
- A piece of code that run in concurrent with other threads
- Each thread is a ordered sequence of instructions
- Threads are being extensively used to express concurrency on both single and multiprocessors machines
- Programming a task having multiple threads of control called – Multithreading or Multithreaded Programming

Java Threads

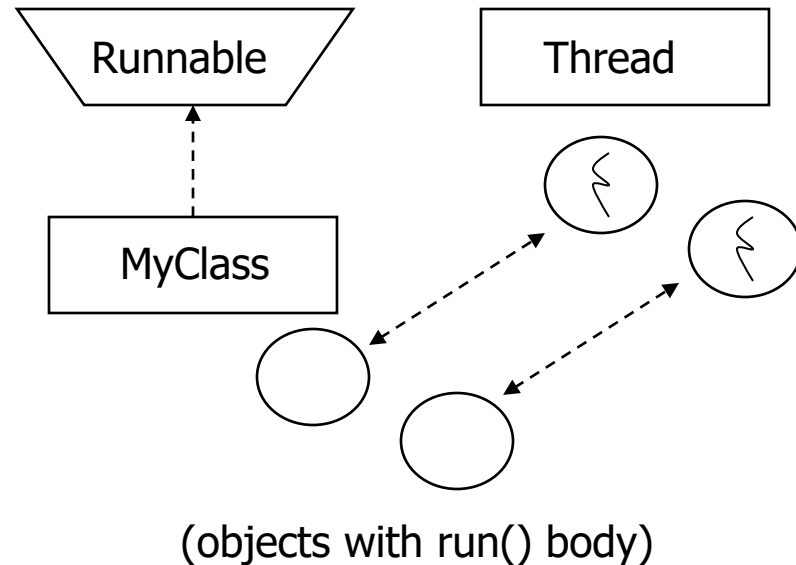
- Java has built in thread support for Multithreading
- Synchronization
- Thread Scheduling
- Inter-Thread Communication:
 - `currentThread` `start` `setPriority`
 - `yield` `run` `getPriority`
 - `sleep` `stop` `suspend`
 - `resume`
- Java Garbage Collector is a low-priority thread

Threading Mechanisms...

- Create a class that extends the Thread class
- Create a class that implements the Runnable interface



[a]



[b]

1st method: Extending Thread class

- Create a class by extending Thread class and override run() method:

```
class MyThread extends Thread
{
    public void run()
    {
        // thread body of execution
    }
}
```

- Create a thread:

```
MyThread thr1 = new MyThread();
```

- Execute a thread (s):

```
thr1.start();
```

- Create and Execute:

```
new MyThread().start();
```

An example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println(" this thread is running ... ");  
    }  
}
```

```
class ThreadEx1 {  
    public static void main(String [] args ) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

2nd method: Threads by implementing Runnable interface

- Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{
    .....
    public void run()
    {
        // thread body of execution
    }
}
```

- Creating Object:

```
MyThread myObject = new MyThread();
```

- Creating Thread Object:

```
Thread thr1 = new Thread( myObject );
```

- Start Execution:

```
thr1.start();
```

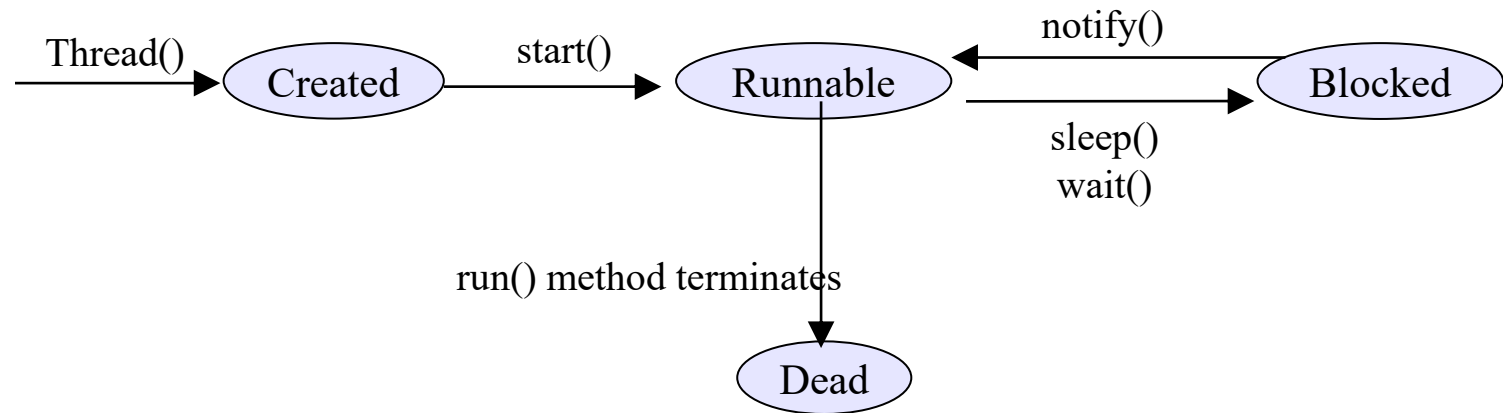
An example

```
class MyThread implements Runnable {  
    public void run() {  
        System.out.println(" this thread is running ... ");  
    }  
}
```

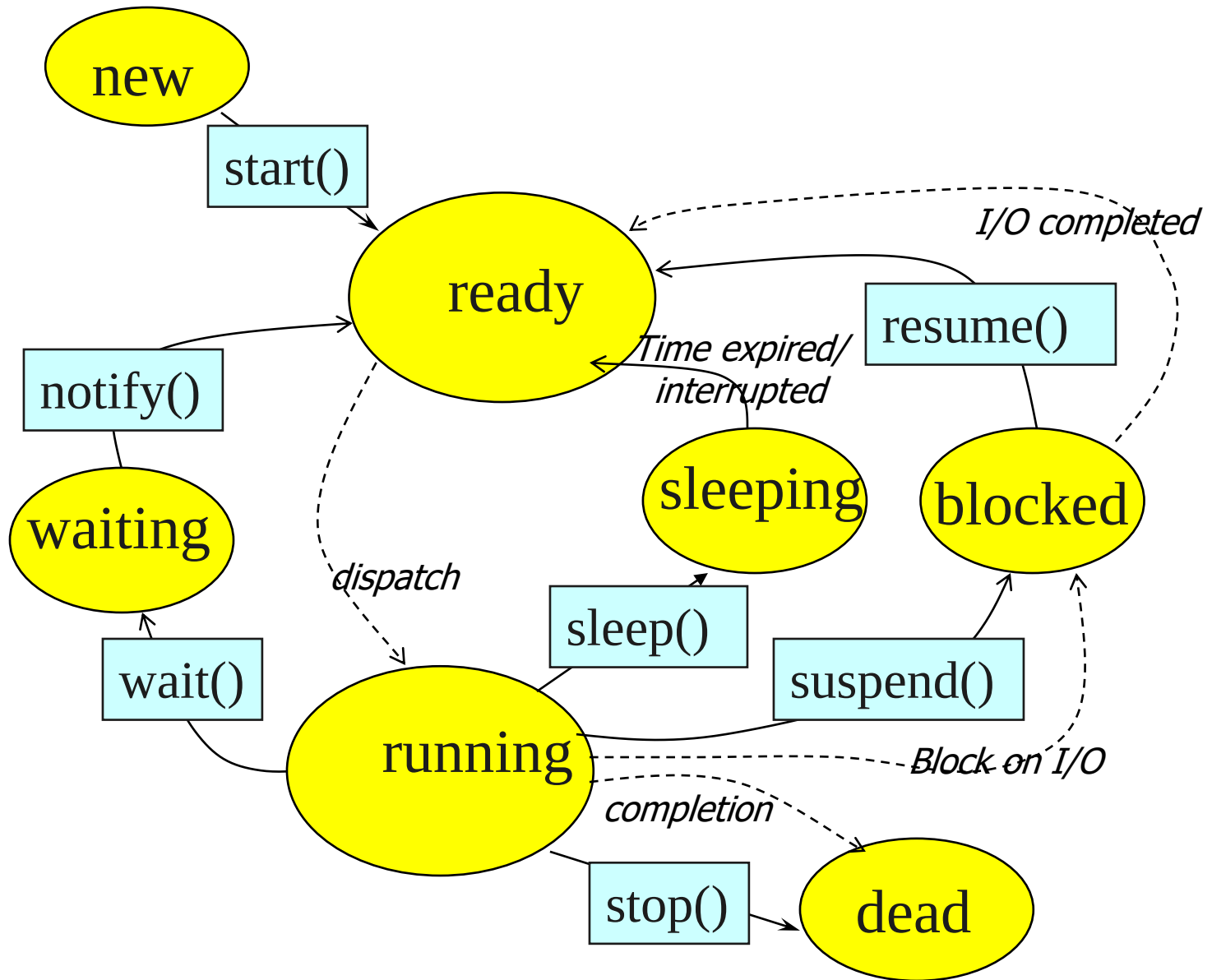
```
class ThreadEx2 {  
    public static void main(String [] args ) {  
        Thread t = new Thread(new MyThread());  
        t.start();  
    }  
}
```

Thread States

- Threads can be in one of four states
 - Created, Running, Blocked, and Dead
- A thread's state changes based on:
 - Control methods such as start, sleep, yield, wait, notify
 - Termination of the run method



Life Cycle of Thread



A Program with Three Java Threads

- Write a program that creates 3 threads

Three threads example

```
■ class A extends Thread
■ {
■     public void run()
■     {
■         for(int i=1;i<=5;i++)
■         {
■             System.out.println("\t From ThreadA: i= "+i);
■         }
■         System.out.println("Exit from A");
■     }
■ }

■ class B extends Thread
■ {
■     public void run()
■     {
■         for(int j=1;j<=5;j++)
■         {
■             System.out.println("\t From ThreadB: j= "+j);
■         }
■         System.out.println("Exit from B");
■     }
■ }
```

```

■ class C extends Thread
■ {
■     public void run()
■     {
■         for(int k=1;k<=5;k++)
■         {
■             System.out.println("\t From ThreadC: k= "+k);
■         }
■
■         System.out.println("Exit from C");
■     } }

■ class ThreadTest
■ {
■     public static void main(String args[])
■     {
■         new A().start();
■         new B().start();
■         new C().start();
■     }
■ }

```

Run 1

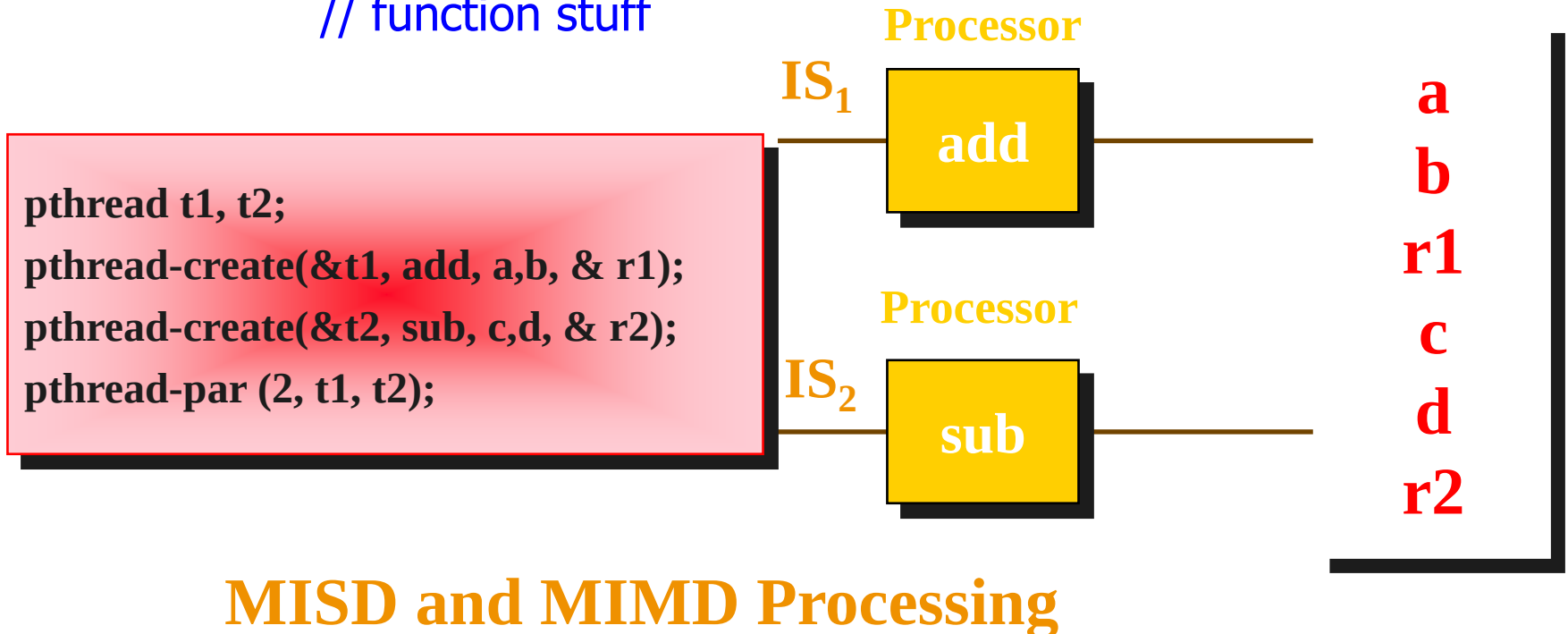
- java ThreadTest
 - From ThreadA: i= 1
 - From ThreadA: i= 2
 - From ThreadA: i= 3
 - From ThreadA: i= 4
 - From ThreadA: i= 5
 - Exit from A
 - From ThreadC: k= 1
 - From ThreadC: k= 2
 - From ThreadC: k= 3
 - From ThreadC: k= 4
 - From ThreadC: k= 5
 - Exit from C
 - From ThreadB: j= 1
 - From ThreadB: j= 2
 - From ThreadB: j= 3
 - From ThreadB: j= 4
 - From ThreadB: j= 5
 - Exit from B

Run2

```
■ java ThreadTest
    From ThreadA: i= 1
    From ThreadA: i= 2
    From ThreadA: i= 3
    From ThreadA: i= 4
    From ThreadA: i= 5
    From ThreadC: k= 1
    From ThreadC: k= 2
    From ThreadC: k= 3
    From ThreadC: k= 4
    From ThreadC: k= 5
Exit from C
    From ThreadB: j= 1
    From ThreadB: j= 2
    From ThreadB: j= 3
    From ThreadB: j= 4
    From ThreadB: j= 5
Exit from B
Exit from A
```

Process Parallelism

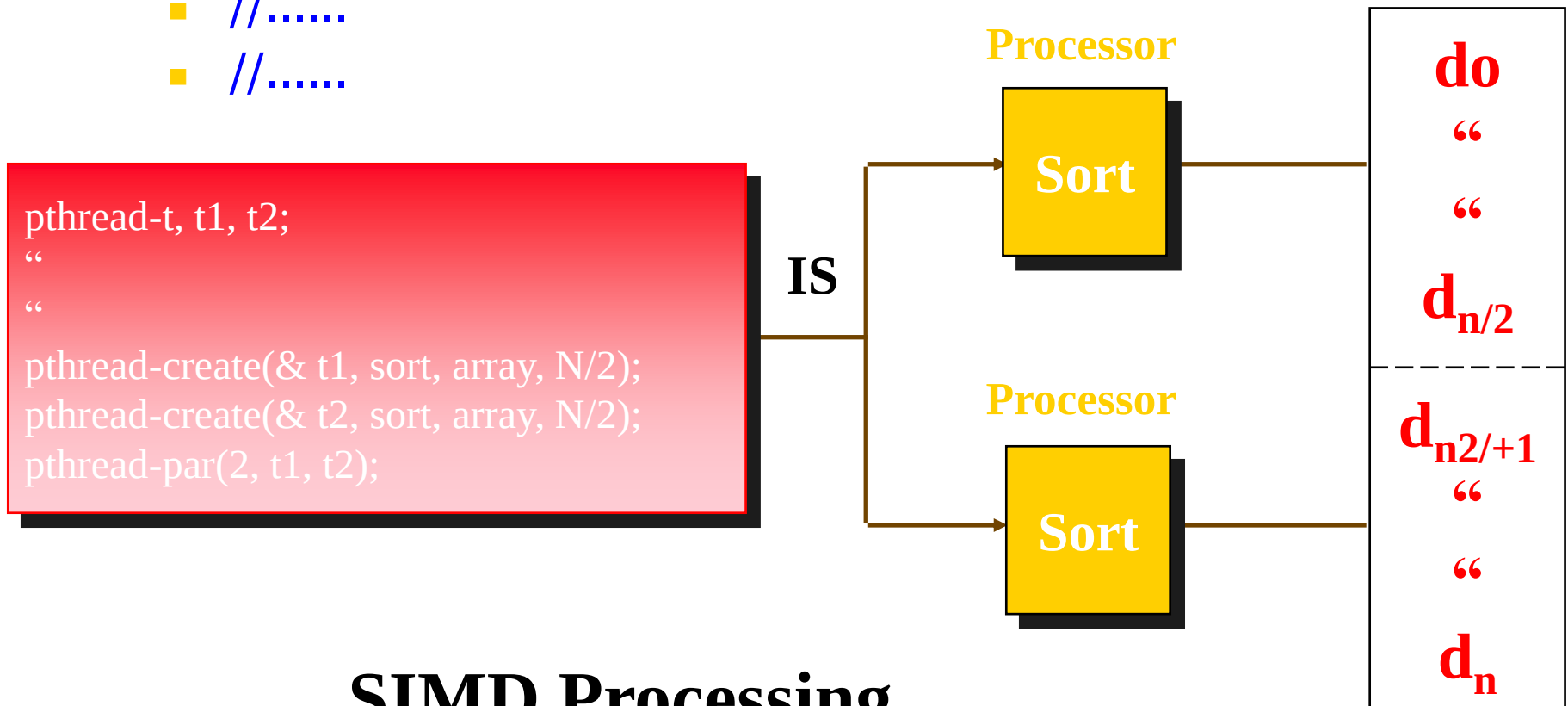
- `int add (int a, int b, int & result)`
 `// function stuff`
- `int sub(int a, int b, int & result)`
 `// function stuff`



Data Parallelism

- `sort( int *array, int count)`
- `//.....`
- `//.....`

Data



SIMD Processing

Thread Priority

- In Java, each thread is assigned priority, which affects the order in which it is scheduled for running. The threads so far had same default priority (NORM_PRIORITY) and they are served using FCFS policy.
 - Java allows users to change priority:
 - ThreadName.setPriority(intNumber)
 - MIN_PRIORITY = 1
 - NORM_PRIORITY=5
 - MAX_PRIORITY=10

Thread Priority Example

```
class A extends Thread
{
    public void run()
    {
        System.out.println("Thread A started");
        for(int i=1;i<=4;i++)
        {
            System.out.println("\t From ThreadA: i= "+i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread
{
    public void run()
    {
        System.out.println("Thread B started");
        for(int j=1;j<=4;j++)
        {
            System.out.println("\t From ThreadB: j= "+j);
        }
        System.out.println("Exit from B");
    }
}
```

Thread Priority Example

```
class C extends Thread
{
    public void run()
    {
        System.out.println("Thread C started");
        for(int k=1;k<=4;k++)
        {
            System.out.println("\t From ThreadC: k= "+k);
        }
        System.out.println("Exit from C");
    }
}

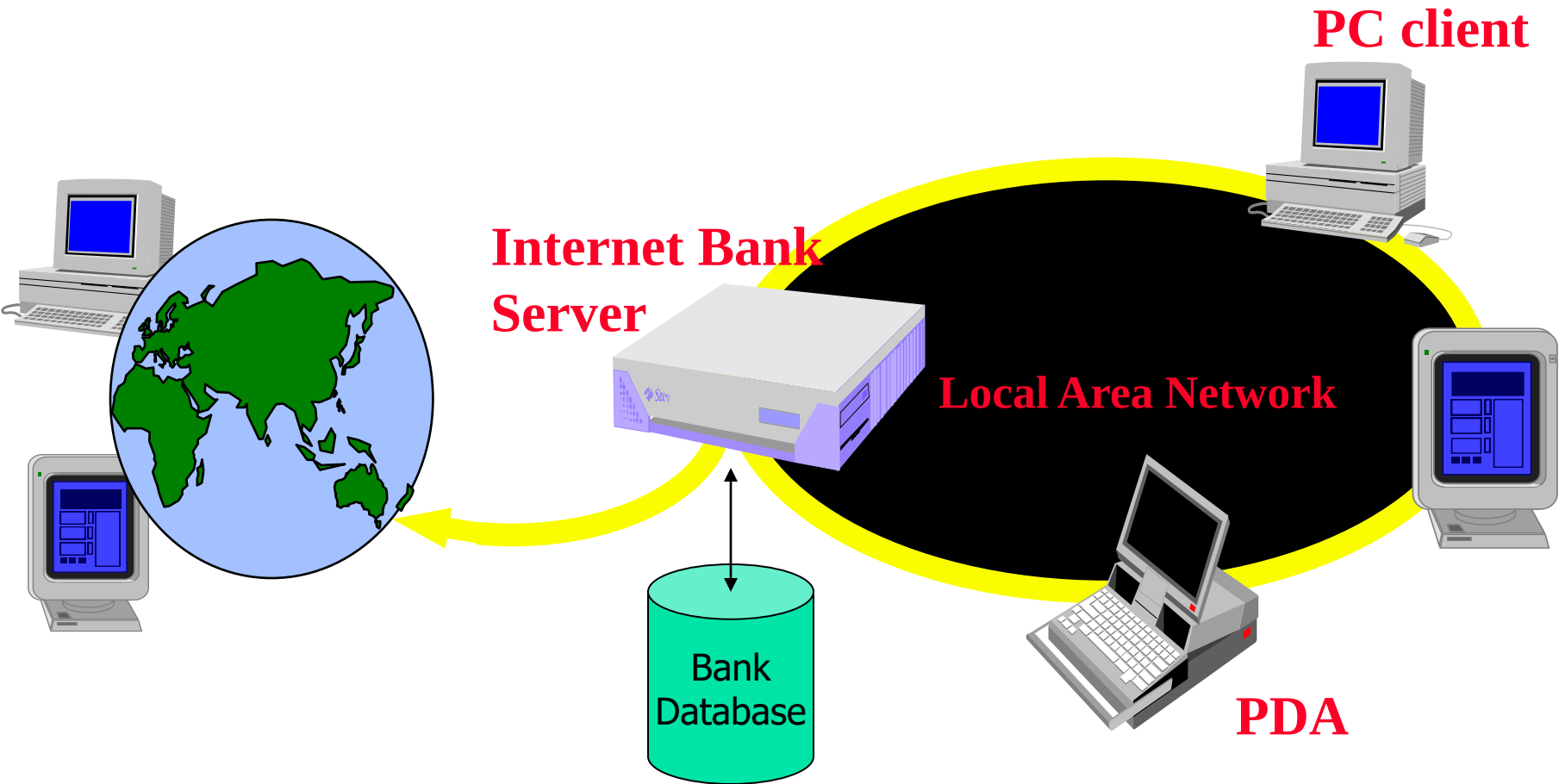
class ThreadPriority
{
    public static void main(String args[])
    {
        A threadA=new A();
        B threadB=new B();
        C threadC=new C();
        threadC.setPriority(Thread.MAX_PRIORITY);
        threadB.setPriority(threadA.getPriority()+1);
        threadA.setPriority(Thread.MIN_PRIORITY);
        System.out.println("Started Thread A");
        threadA.start();
        System.out.println("Started Thread B");
        threadB.start();
        System.out.println("Started Thread C");
        threadC.start();
        System.out.println("End of main thread");
    }
}
```

```
C:\java>java ThreadPriority
Started Thread A
Started Thread B
Thread A started
Thread B started
Started Thread C
    From ThreadB: j= 1
    From ThreadA: i= 1
Thread C started
    From ThreadB: j= 2
End of main thread
    From ThreadB: j= 3
    From ThreadC: k= 1
    From ThreadA: i= 2
    From ThreadC: k= 2
    From ThreadC: k= 3
    From ThreadB: j= 4
    From ThreadC: k= 4
    From ThreadA: i= 3
Exit from C
Exit from B
    From ThreadA: i= 4
Exit from A
```

Accessing Shared Resources

- Applications Access to Shared Resources need to be coordinated.
 - Ex1: Printer (two person jobs cannot be printed at the same time)
 - Ex2: Simultaneous operations on your bank account.
 - Can the following operations be done at the same time on the same account?
 - Deposit()
 - Withdraw()
 - Enquire()

Online Bank: Serving Many Customers and Operations



Concurrent Access to Data

- Concurrent access to data can lead to data integrity problems
 - If two sources attempt to update the same data at the same time, the result of the data can be undefined.
 - The outcome is determined by how the scheduler schedules the two sources.
 - Since the schedulers activities cannot be predicted, the outcome cannot be predicted
- Databases deal with this mechanism through "**locking**"
 - If a source is going to update a table or record, it can lock the table or record until the time that the data has been successfully updated.
 - While locked, all access are blocked except to the source which holds the lock.
- Java has the equivalent mechanism. It is called **synchronization**
 - Java has a keyword called **synchronized**

Shared Resources

- If one thread tries to read the data and other thread tries to update the same data, it leads to inconsistent state
- This can be prevented by synchronising access to the data
- Use “Synchronized” method:

```
public synchronized void update()  
{  
    ...  
}
```

3 Threads sharing the same object

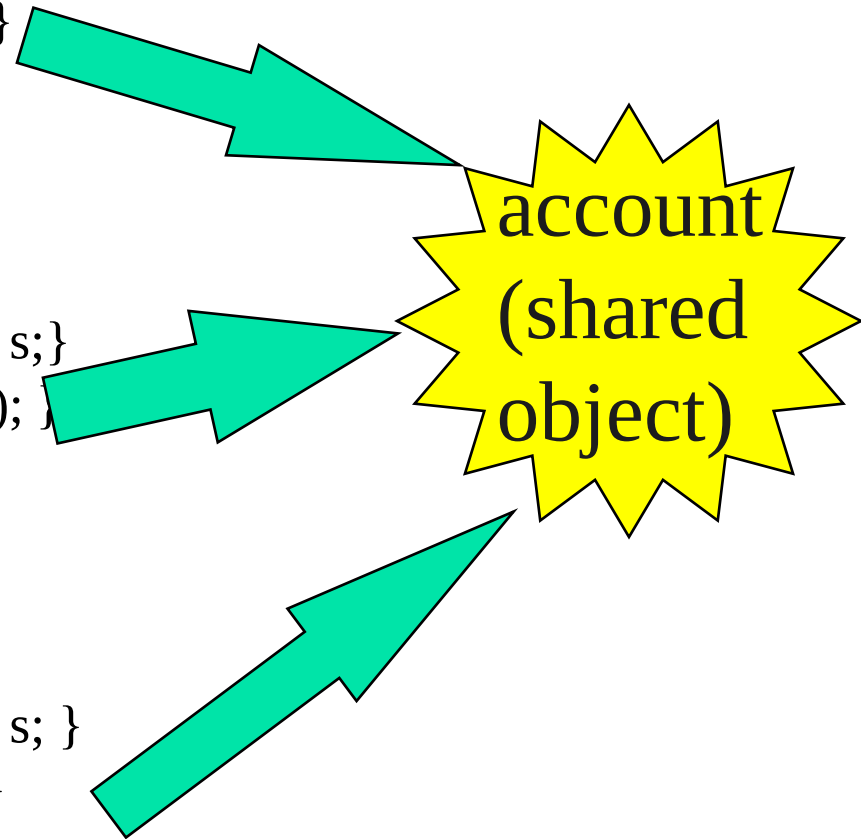
```
class InternetBankingSystem
{
    public static void main(String [] args )
    {
        Account accountObject = new Account ();
        Thread t1 = new Thread(new Thread1(accountObject));
        Thread t2 = new Thread(new Thread2(accountObject));
        Thread t3 = new Thread(new Thread3(accountObject));
        t1.start();
        t2.start();
        t3.start();
        // DO some other operation
    } // end main()
}
```

Shared account object between 3 threads

```
class Thread1 implements Runnable {  
    Account account;  
    public Thread1 (Account s) { account = s;}  
    public void run() { account.deposit(); }  
} // end class MyThread
```

```
class Thread2 implements Runnable {  
    Account account;  
    public Thread2 (Account s) { account = s;}  
    public void run() { account.withdraw(); }  
} // end class YourThread
```

```
class Thread3 implements Runnable {  
    Account account;  
    public Thread3 (Account s) { account = s; }  
    public void run() {account.enquire(); }  
} // end class HerThread
```



Monitor (shared object access): serializes operation on shared object

```
class Account { // the 'monitor'
    int balance;

    // if 'synchronized' is removed, the outcome is unpredictable
    public synchronized void deposit( ) { // METHOD BODY :
        balance = balance + deposit_amount;
    }

    public synchronized void withdraw( ) { // METHOD BODY:
        balance = balance - deposit_amount;
    }

    public synchronized void enquire( ) { // METHOD BODY:
        display balance.
    }
}
```

Properly Terminating Threads

- In Java 1.1, the Thread class had a stop() method
 - One thread could terminate another by invoking its stop() method.
 - However, using stop() could lead to deadlocks
 - The stop() method is now not used. **DO NOT use the stop method to terminate a thread**
- The correct way to stop a thread is to have the run method terminate
 - Add a boolean variable which indicates whether the thread should continue or not
 - Provide a set method for that variable which can be invoked by another thread

Terminating Thread Example

```
public class Test implements Runnable
{
    private Thread theThread;
    private boolean stopThread = false;

    public void start()
    {
        if (theThread == null)
        {
            theThread = new Thread(this);
            theThread.start();
        }
    }

    public void setStopThread(boolean aValue)
    {
        stopThread = aValue;
    }

    public void run()
    {
        while(true)
        {
            if (stopThread)
                break;

            // ....
        }
    }
}
```

Yield() and Sleep()

- Sometimes a thread can determine that it has nothing to do
 - Sometimes the system can determine this, i.e., waiting for I/O
- **When a thread has nothing to do, it should not use CPU**
 - This is called a **busy-wait**, i.e., busy using CPU doing nothing
 - threads in busy-wait are continually checking if there is anything to do.
- It is worthwhile to run a CPU monitor program on your desktop
 - You can see that a thread is in busy-wait when the CPU monitor goes up (usually to 100%), but the application doesn't seem to be doing anything.
- Threads in busy-wait should be moved from the Run queue to the Wait queue so that they do not block the CPU
 - Use **yield()** or **sleep(time)**
 - **yield** simply tells the scheduler to schedule another thread
 - **sleep** guarantees that this thread will remain in the wait queue for the specified number of **milliseconds**.